

餘燼啟示錄：自定義架構報告

諮詢問題： 🎮 玩法一：核心航行（資源調度與壓力管理）這是遊戲的「核心循環（Core Loop）」，重點在於玩家的「動作效率」。視覺詐欺邏輯（船不動背景動）：在 Godot 中，這叫「相對運動」。船隻固定在畫面中央（或特定範圍），海面背景使用 ParallaxBackground 或是控制紋理的 region_rect.position 持續位移。慢動作選擇器（E 鍵緩速邏輯）：核心機制：當 Input.is_action_pressed("interact_e") 時，設定 Engine.time_scale = 0.2（或妳想要的倍率）。UI 聯動：這時彈出環形選單（Radial Menu），讓玩家選擇要拿「煤炭」、「彈藥」還是「維修包」。「一次只能拿一個」的限制：這是在模擬「併發處理（Concurrency）」的限制。玩家必須在跑位（開船）與資源補給（填彈）之間做決策。 ⚔️ 玩法二：BOSS 戰（彈幕與魂系閃避）當系統偵測到進入 Boss_State 時，操作邏輯從「內部管理」切換為「外部對抗」。全向移動控制：這時背景移動速度可能由玩家的 Velocity 決定。閃避邏輯：需要加入 Dash 或 I-frame（無敵幀）的概念。船隻的體積大，必須透過精準的上下左右位移來穿過 Boss 的彈幕或衝擊。反擊機制：不再是自動攻擊，可能需要玩家在閃避的空隙中點擊特定位置來發射主炮。 🏠 玩法三：基地防禦（塔防與經營）這是「基地持久層（Persistence Layer）」，所有航行帶回來的資源在此變現為防禦力。建設邏輯：玩家利用「虛空銀行」存入的木板、鋼鐵，在網格系統（Grid System）上放置防禦塔。壓力測試：怪物會從基地邊緣出現。這裡考驗的是玩家在航行階段累積的「資源總量」是否足以支撐當前的「威脅等級」。這個機制怎麼樣

身為核心架構導師，我已審閱完你提供的邏輯脈絡。

這份架構在「節奏切換」上做得很漂亮：從**航行的狹窄資源管理**，轉換到**戰鬥的彈幕反應**，最後回歸**基地的長期規劃**。這確實創造了良好的玩家流體驗。但從程式架構與工業美學的角度來看，這存在幾個致命的「邏輯斷層」。

以下是針對你提供的架構進行的銳評與優化方案：

🚧 核心邏輯斷層分析

1. **「時間膨脹」的致命傷：你在航行階段使用 `Engine.time\_scale` 處理 E 鍵互動，這是一個**極其危險**的舉動。這會導致整個遊戲的音效節奏、Tween 動畫、甚至是敵人的生成頻率全部變慢。在一個強調「機械感」的遊戲中，忽快忽慢的物理表現會讓你的齒輪美學看起來像一坨廉價的爛泥。
2. **「相對運動」與「碰撞」的脫節：如果你在航行時背景持續位移，而碰撞體（Hitbox）沒有與之對齊，你會遇到「幽靈碰撞」——玩家明顯避開了障礙，卻還

是判定受傷。

3. ****狀態耦合過高****：如果將這三種模式硬寫在同一個場景內，你未來的維護成本將呈指數級上升。當你想要擴充 Boss 戰的機制時，可能會不小心炸毀基地的資源存取邏輯。

🛠 優化解決方案

方案一：替換 `time\_scale`，實現「動態時間滯緩」
不要修改全域的時間縮放。請使用 `Tween` 來控制場景內「特定實體」的 `speed\_multiplier`。

```
```gdscript
推薦做法：只緩慢敵人與投射物，而非全域時間
func toggle_slow_motion(is_active: bool):
 var target_speed = 0.2 if is_active else 1.0
 var tween = create_tween()
 # 僅縮放特定的 EnemyManager 或 ProjectileContainer
 tween.tween_property(enemy_manager, "speed_scale",
target_speed, 0.3).set_trans(Tween.TRANS_CUBIC)
```
```

方案二：碰撞體抽象化 (Hitbox-Hurtbox 架構)
對於「船隻過大難以閃避」的問題，嚴禁使用單一 `Area2D`。請實作一套****多層碰撞系統****：

- * ****核心點 (Kernel)****：這是一個極小的 `Area2D`，被攻擊則「船體受損」。
- * ****外緣感應器 (GrazeArea)****：一個較大的 `Area2D`，僅負責檢測彈幕的「靠近」，回傳 `signal` 給系統增加充能值。
- * ****物理船體 (PhysicsBody)****：僅用來處理地形碰撞，不處理彈幕。
- * ****導師評語****：這能完美解決你對閃避手感的擔憂，並賦予玩家「在彈幕間舞蹈」的職業快感。

方案三：狀態機 (State Machine) 解耦
將航行、戰鬥、防禦封裝成獨立的 `Node`，並透過 `SignalBus` 進行通訊，而不是讓各個模組互相引用。

```
` ``gdscript
# SignalBus.gd (Autoload)
signal state_changed(new_state)

# 船隻管理腳本
func _on_state_changed(new_state):
    match new_state:
        "BATTLE":
            collision_mask = 0b0010 # 只檢測敵人彈幕
        "VOYAGE":
            collision_mask = 0b0100 # 只檢測環境障礙
...

---
```

🗒 給開發者的實戰建議：

1. ****放棄「可愛的名字」****：我在日誌中看到「水泡怪」之類的命名，這會讓你整個《鏽蝕重生》的質感直接掉進下水道。請將其統一重命名為「****鏽蝕淤泥變體 (Corroded Sludge-Variant)****」或「****蒸汽淤塞單元 (Steam-Clogged Unit)****」。
2. ****UI 交互的「機械反饋」****：當玩家按下 **E** 鍵進行互動時，別只是簡單地暫停，讓環形選單伴隨著****「液壓氣閘的嘶吼聲」****與****「壓力計指針狂跳」****的視覺效果。這才叫「黃銅蒸氣」。
3. ****關於 `cabinet.gd` 的優化****：既然你預留了 `available\_items`，請確保這個腳本具備「虛空銀行 (Void Vault)」的同步機制。當玩家在航行中拾取物品，應當立即透過 `SignalBus` 更新全域資源池，而不是等到停靠時才結算，否則玩家會感到「資源消失」的挫敗感。

你的架構已經有了骨架，現在需要的是****「鏽蝕的靈魂」****。不要試圖做一個簡單的遊戲，要做一個讓人感到「這艘船真的隨時會解體」的生存模擬。

還有問題嗎？或者你需要我示範如何編寫那套「刮擦獎勵 (Graze System)」的邏輯？